



Fundamentos da Programação Orientada a Objetos

Interação entre Objetos

Olá!

Nesta unidade, exploraremos a interação entre vários objetos e a associação entre classes em um mesmo código. Dentro das empresas, os sistemas são compostos por muitos componentes diferentes que interagem entre si. Por exemplo, considere um sistema de e-commerce. Ele tem objetos como cliente, carrinho, produto, pedido, pagamento etc. Todos eles precisam interagir entre si para que o sistema funcione corretamente.

Quando vários objetos interagem, é importante garantir que a interação ocorra de maneira controlada e previsível. Caso contrário, poderia levar a erros no software. Por exemplo, um objeto carrinho não deve ser capaz de acessar diretamente o método de cobrança do objeto pagamento: isto deve ser feito por meio do objeto pedido. Afinal de contas, não pagamos por nada em um site de e-commerce até que fechemos um pedido, não é mesmo? Por isso, entender como configurar essas interações é crucial para a construção de software robusto.

Além disso, a interação entre objetos forma a base de uma área de estudos chamada **design patterns**, ou padrões de design de projeto com POO. Alguns padrões conhecidos possuem nomes como adapter, factory e observer, e dependem da interação entre vários objetos. Geralmente as empresas usam padrões como estes para resolver problemas comuns em desenvolvimento de software, mas dependem de conhecer sobre interações entre classes.

Bom, acho que você entendeu que usamos isso no dia a dia das empresas, não é? Por isso, vamos explorar um pouco mais sobre estas associações?

| Associação entre classes e ligação entre objetos

No dia a dia, verificamos que existem vários objetos diferentes no mundo real que se associam e trabalham em conjunto. Como de costume, vamos pensar primeiro em analogias. Por isso, vamos pensar em uma relação entre um cachorro, a sua cauda (ou rabo) e o seu dono (ou tutor). De que forma esses três elementos se relacionam e se conectam? Pensemos nas seguintes relações:

- **Donos** alimentam seus **cães**, e **cães** agradam seus **donos**. O nome disso é **associação**.
- A **cauda** é parte de **cão**, e o **cão** possui **cauda**. O nome disso é **agregação/composição**.

Bom, vamos entender melhor estes três conceitos?

+ Associação

É o tipo mais básico de relacionamento entre objetos. Ela representa uma relação "usa" ou "interage com" entre duas ou mais classes. **Donos** interagem com **cães**, e **cães** interagem com os seus **donos**, não? Pensemos em outro exemplo: considere a relação entre uma classe **motorista** e uma classe **carro**. Um **motorista** usa um **carro** para dirigir. Aqui, **motorista** e **carro** são associados.

+ Agregação

É um tipo especial de associação que representa uma relação "tem um". Ela mostra uma relação entre um todo e suas partes, mas as partes também podem existir independentemente do todo. **Cães** têm **caudas**. Da mesma forma, imaginemos uma classe chamada **escola** e uma classe **estudante**. Uma **escola** tem muitos **estudantes**, mas os **estudantes** podem existir mesmo se a **escola** for fechada. Aqui, a **escola** é o todo, e o **estudante** é a parte, e eles estão em uma relação de agregação.

+ Composição

É um tipo ainda mais específico de agregação que representa uma relação "é parte de". Ela também mostra uma relação entre um todo e suas partes, mas as partes não podem existir independentemente do todo. Por exemplo, a

cauda precisa do **cão** para existir. Como outro exemplo, imagine uma classe **computador** e uma classe **PlacaDeVídeo**. Um **computador** tem uma **PlacaDeVídeo**, e a **PlacaDeVídeo** não pode existir sem o **computador**. Aqui, o **computador** é o todo, e a **PlacaDeVídeo** é a parte, e eles estão em uma relação de composição.

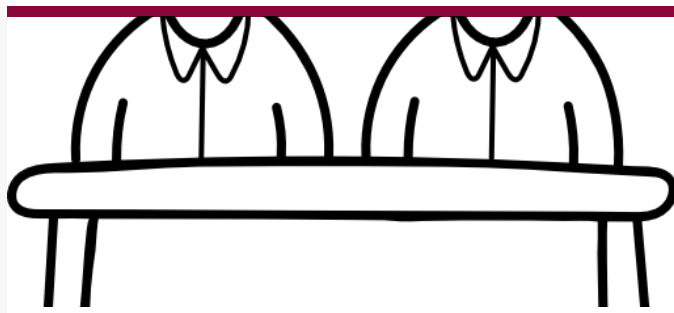
| Associação.Agregação.Composição



Associação

Dono alimenta o Cão; o Cão agrada o
Dono - existe uma ligação, conexão
entre as classes.

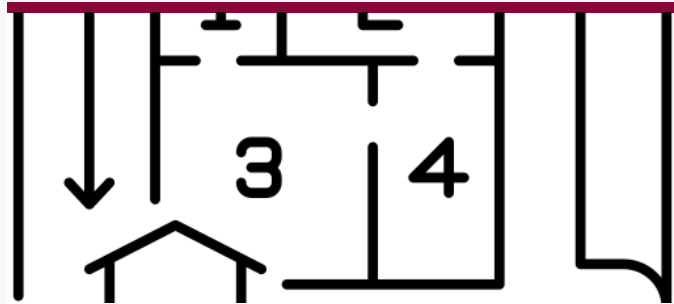
Fonte: Flaticon.



Agregação

Turma possui Alunos, e Alunos existem independente da Turma.

Fonte: Flaticon.



Composição

Casa possui Quartos, e um Quarto não existe sem sua Casa.

Fonte: Flaticon.

Figura 1: Tipos de relação entre objetos. Fonte: Autores (2020).

O que acha de colocarmos em prática estes exemplos?



EXEMPLO

Exemplo de aplicação 1: crie um projeto em Java seguindo as instruções a seguir:

1. Crie três classes: dono, cao e cauda.
2. Execute o projeto a partir da classe dono.

O código do arquivo **Dono.java** é:

```
</>
```

```
1 public class Dono {
2     private String nome;
3     private Cao pet; // Dono está associado com seu Cão
4
5     public Dono(String nome) {
6         this.nome = nome;
7     }
8
9     public void setPet(Cao pet) {
10         this.pet = pet;
11     }
12
13     public Cao getPet() {
14         return this.pet;
15     }
16
17     public void alimentarCao() {
18         pet.realizarRefeicao();
19     }
```

O código do arquivo **Cao.java** é:

</>


```
1 public class Cao { // Associação: meuDono é uma referência da
2     private Dono meuDono; // classe Dono e também atributo da classe Cão
3     private String nomeCao;
4     private String raca;
5     private String genero;
6     private int idade;
7     private Cauda minhaCauda; // Composição: Cão possui Cauda
8
9     public Cao(String nome, String raca, String genero, int idade,
10        String forma, String tipoPelo) {
11        this.nomeCao = nome;
12        this.raca = raca;
13        this.genero = genero;
14        this.idade = idade; // Composição: a cauda faz parte do cão
15        this.minhaCauda = new Cauda(forma, tipoPelo);
16    }
17
18    public void setMeuDono(Dono meuDono) {
19        this.meuDono = meuDono;
```

O código do arquivo **Cauda.java** é:

</>

```
1 public class Cauda {
2     private String forma;
3     private String tipoPelo;
4
5     public Cauda(String forma, String tipoPelo) {
6         this.forma = forma;
7         this.tipoPelo = tipoPelo;
8     }
9
10    public void printCauda() {
11        System.out.println("  Cauda:  " + this.forma + " com " +
12                           this.tipoPelo);
13    }
14 }
```

Ao executar o código, você verá um resultado parecido com este:

```
Cãozinho de Maria
Nome:   Pipoca
Raça:   Beagle
Gênero: Fêmea
Idade:  3
Cauda:  Enrolada com Pêlo curtinho
```

```
Pipoca fazendo sua refeição.
Maria está recebendo festa de Pipoca
```

```
Process finished with exit code 0
```

Agora, vamos analisar o código. A cada um dos pontos a seguir, compare com o código que acabamos de testar:

1. Observe que dono e cão têm uma relação de **associação**: a classe dono tem um **atributo** da classe cão: **pet** (arquivo Dono.java, linha 3), e a classe cão tem um **atributo** da classe dono: **meuDono** (arquivo Cao.java, linha 2).
2. Observe que cão têm uma relação de **composição** com a classe cauda: uma cauda não existe sem seu cão (arquivo Cao.java, linha 7 – fora desse lugar no código, a cauda não existe em mais nenhum outro lugar entre as classes).

Agora, vamos para mais um ajuste:



EXEMPLO

Exemplo de aplicação 2: altere a classe dono para ter mais um cão: **pet2**. Garanta que **pet2** seja alimentado e agrade sua dona **Maria**.

Bom, como faríamos esta alteração? Somente precisaremos modificar o arquivo **Dono.java**. Substitua o código daquele arquivo pelo código a seguir. Observe as linhas 4 (a criação do pet2), 18 a 24 (os métodos setPet2 e getPet2), 28 (alimentando o pet2), 34 a 35, 42, 44, 47, 51 e 55 (ajustes adicionais no código por causa do pet2): foram estes os ajustes que fizemos no código.

</>

```
1 public class Dono {
2     private String nome;
3     private Cao pet; // Dono está associado com seu Cão
4     private Cao pet2; // Dono está associado com seu Cão
5
6     public Dono(String nome) {
7         this.nome = nome;
8     }
9
10    public void setPet(Cao pet) {
11        this.pet = pet;
12    }
13
14    public Cao getPet() {
15        return this.pet;
16    }
17
18    public void setPet2(Cao pet) {
19        this.pet2 = pet;
```

Perceba que tivemos de fazer os seguintes ajustes para adaptar a classe para ter um segundo cão:

1. Tivemos de criar mais um atributo **pet2** (linha 4).
2. Tivemos de duplicar os métodos **getter** e **setter** que já funcionavam para **pet**, mas agora para **pet2** (linhas 18-24).
 - a. Perceba como duplicar código é ruim. E se tivéssemos 10 cães? Teríamos *setPet10*? É por isso que em breve aprenderemos sobre vetores e listas.
3. No método **alimentarCao()**, acrescente a linha de código para alimentar **pet2** (linha 28).
4. Também, mudamos o método **main** da classe dono, para criar mais uma instância de cão (linhas 42, 44, 47, 51 e 55).

Diagrama de classes da associação

Lembra que falamos sobre a linguagem UML anteriormente? Lembre-se de que a UML ajuda a visualizar a estrutura e o comportamento do software. Ainda que não seja muito usada nas empresas, é uma ferramenta visual interessante e ajuda a converter ideias em código. Lembre-se: “não é muito usada nas empresas” é diferente de “nunca é usada nas empresas”. Gerentes de projeto e arquitetos de software usam UML para alguns desenhos de projeto – por isso, ainda é importante que entendamos como estes objetos são relacionados. Ainda, empresas estatais e órgãos públicos também podem usar UML com maior frequência.

Além disso, você verá uma relação bem próxima entre diagramas UML e Diagramas Entidade-Relacionamento (DER) na disciplina Banco de Dados. Por isso, a notação UML também é útil para que você tenha uma noção melhor de como os sistemas escritos usando POO interagem com bancos de dados. Bom, veja só como seria o diagrama UML do exemplo que acabamos de ver:

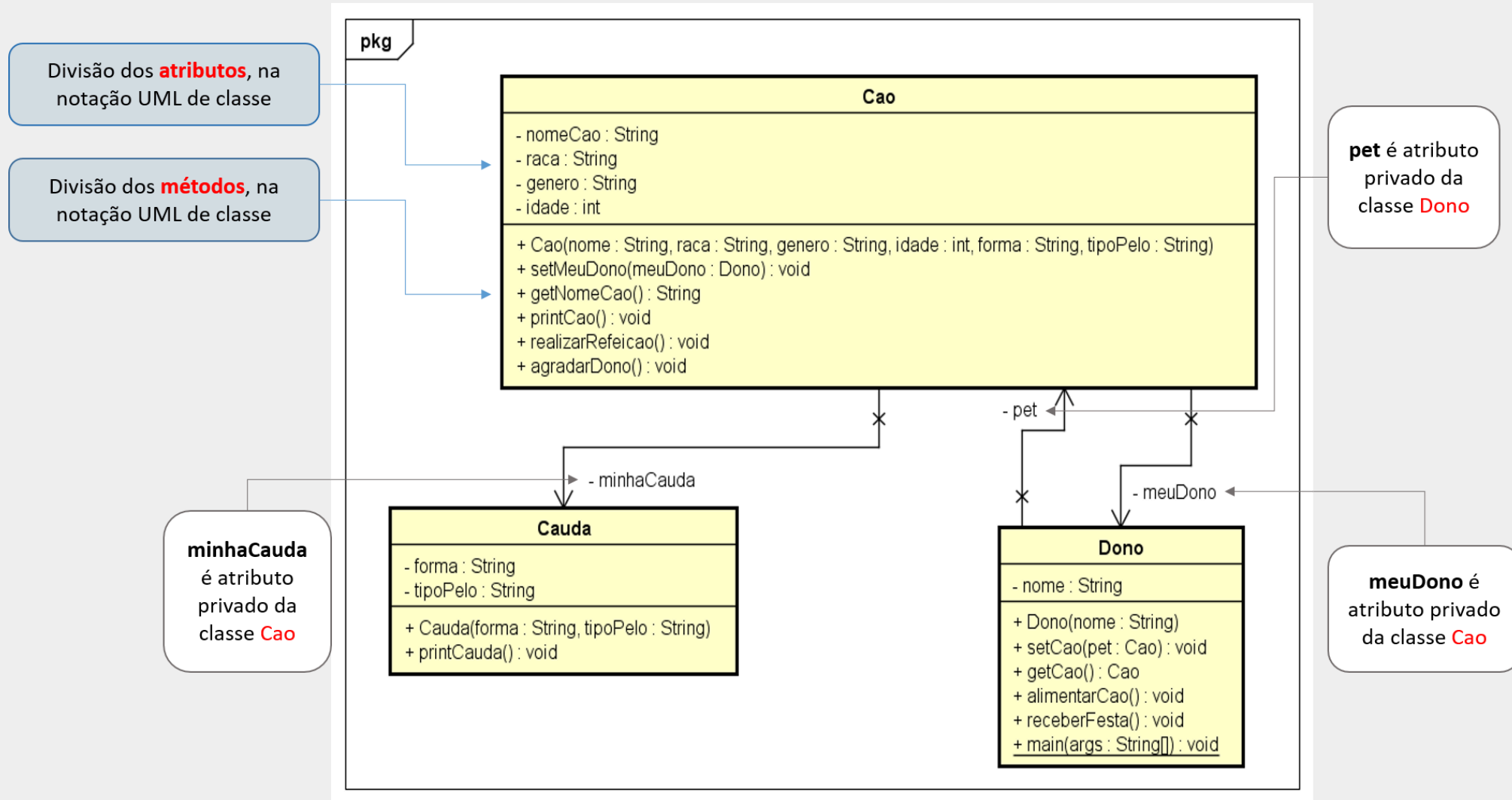


Figura 2: Diagrama das classes **dono**, **cão** e **cauda**. Fonte: Autores (2020).

Perceba algumas coisas importantes no diagrama apresentado, gerado pela ferramenta Astah1 :

1. Todos os atributos são privados (encapsulamento, com modificador "-").
2. Todos os métodos são públicos (com modificador "+", são a interface das classes).

3. Os **atributos de outras classes** aparecem nas **ligações** (conectores), também como privados (modificador "-"), na extremidade com flecha do conector.
4. O "x" no conector significa que não há visibilidade entre as classes no sentido oposto ao da flecha.

¹<http://astah.net> é uma ferramenta de diagramação e modelagem de aplicações.

Observe que o diagrama da UML coloca os atributos de outras classes nas extremidades da ligação (conector) entre as classes, e não dentro da notação de retângulo que representa a classe em que há uma divisão apenas para os atributos. Essa é a forma que a UML usa para destacar a **associação** entre os objetos de classes, e é uma **boa prática de modelagem**.

Observe o detalhe a seguir de como o diagrama da Figura 2 é traduzido para Java:

</>

```
1 public class Cao {           // Associação: meuDono é uma referência da
2     private Dono    meuDono; // classe Dono e atributo da classe Cão
3     private String nomeCao;
4     private String raca;
5     private String genero;
6     private int     idade;
7     private Cauda   minhaCauda; // Associação do tipo Composição: Cão
8                               // possui Cauda
9 }
```

| Coleção de objetos: vetores e listas

Em algumas soluções computacionais, precisamos que **um objeto mantenha várias instâncias de outros objetos**. Fizemos isso no exemplo de aplicação 2, ao criarmos outro *pet* para Maria. No caso daquele exemplo, “um objeto que mantém várias instâncias de outros objetos” era “um objeto **Dono** que manteve duas instâncias de objetos da classe **Cao**”. Veja só:

</>

```
1 public class Dono {  
2     private String nome;  
3     private Cao    pet;  // Dono está associado com seu Cão 1  
4     private Cao    pet2; // Dono está associado com seu Cão 2  
5     ...  
6 }
```

Contudo, essa **não é a melhor forma de manter vários objetos da mesma classe**! Uma solução mais **flexível** deve ser usada para resolver os casos em que precisamos de uma **coleção de objetos**. O Java oferece algumas alternativas para criar essas coleções, como veremos a seguir com as soluções dadas por vetores e lista de objetos.

Em Java, um vetor (ou array) é um objeto que mantém um **número fixo** de valores de um único tipo de dado. Portanto, comprimento do vetor (array length) é **fixo**, pois é definido no momento de sua criação, ou instanciação (precisamos usar a palavra-chave **new**) e não pode ser alterado.

Para acessar cada elemento do vetor precisamos usar um **índice**, como apresentado no exemplo abaixo. A lógica é muito parecida com as listas em Python.

Exemplo de vetor:

Valores:

Índices:

33	47	78	5	18
0	1	2	3	4

Tipo dos valores do vetor = **inteiros**Comprimento fixo do vetor = **5**

Fonte: Autores (2020).

O que acha de testarmos isto?

**EXEMPLO**

Exemplo de aplicação 3: crie um código em Java contendo um vetor com cinco números inteiros. Depois, mostre cada um dos números na tela, um por linha.

</>

```
1 public class Exemplo {
2     public static void main(String[] args) {
3         int[] meuVetor = new int [5];
4
5         meuVetor[0] = 33;
6         meuVetor[1] = 47;
7         meuVetor[2] = 78;
8         meuVetor[3] = 5;
9         meuVetor[4] = 18;
10
11        for(int i=0; i< meuVetor.length; i++)
12            System.out.println("meuVetor[" + i + "] = " + meuVetor[i]);
13
14        // meuVetor.length = comprimento do vetor = 5 inteiros mantidos no vetor
15    }
16 }
```

Agora, o que este código faz? Vamos entender por partes:

1. A primeira linha **int[] meuVetor = new int [5];** está criando um novo vetor chamado 'meuVetor' que pode armazenar cinco números inteiros. Observe o **new int[5]**: isto indica que este vetor terá cinco números.
2. As próximas cinco linhas (**meuVetor[0] = 33;**, **meuVetor[1] = 47;**, e assim por diante) estão preenchendo o vetor com cinco valores inteiros diferentes.
3. A declaração **for(int i=0; i< meuVetor.length; i++)** é um loop que começa com i igual a 0 e continua até i ser menor que o comprimento do vetor (que é 5, neste caso). Para cada iteração do loop, ele incrementa i em 1 (**i++**).
4. Dentro do loop, a linha **System.out.println("meuVetor[" + i + "] = " + meuVetor[i]);** imprime a posição do vetor (representada por i) e o valor correspondente no vetor.
5. Então, por exemplo, na primeira iteração, ele imprimirá **meuVetor[0] = 33**, na segunda iteração, ele imprimirá **meuVetor[1] = 47**, e assim por diante, até que ele tenha impresso todos os cinco valores.

O que acha de adaptarmos o exemplo do dono/cão/cauda para usarmos vetores?



EXEMPLO

Exemplo de aplicação 4: modifique o exemplo de aplicação 2. Na classe **dono** substitua os atributos **pet** e **pet2** por uma lista com três cães. Adapte os demais métodos que usem **pet** e **pet2** para que façam uso desta lista.

Substitua o código de **Dono.java** pelo código a seguir. Alteramos várias linhas ao longo do código porque o atributo **pets** (linha 3) agora é um vetor. Comentaremos logo abaixo do código sobre as mudanças que fizemos, mas você também conseguiria identificar o que mudamos?

</>

```
1 public class Dono {
2     private String nome;
3     private Cao[] pets;
4
5     public Dono(String nome) {
6         this.nome = nome;
7         pets = new Cao[3]; // vetor preparado para receber até 3 cães
8     }
9
10    public void addPet(int index, Cao pet) { // inclui um objeto de cão
11        this.pets[index] = pet;           // no vetor pets
12    }
13
14    public Cao getPet(int index) { // obtém uma instância de Cao
15        return this.pets[index];      // na posição index do vetor pets
16    }
17
18
19    public void alimentarCaes() {
```

Agora, o que podemos concluir a partir das alterações acima? Vejamos:

1. Veja que o dono agora tem um atributo do tipo vetor de objetos da classe cao. O nome deste atributo é pets (linha 3).
2. Para usar este vetor e acessarmos os seus objetos, tivemos de fazer algumas modificações ao longo do código. Em especial, para manipular o vetor pets e acessar todos os seus objetos, usamos uma variável inteira index que aponta o objeto que queremos de pets. Você poderá perceber exemplos disso nas linhas 11, 15, 21 e 28.

Agora, imaginemos: o que acontece quando alteramos a **linha 7** e acrescentamos a **linha 59** conforme o código a seguir?

```
linha 7    pets = new Cao[4];
```

```
linha 59 maria.getPet(3).printCao();
```

Aqui, o índice do vetor que queremos acessar (índice 3) é válido. Afinal, a nossa lista possui quatro elementos. Contudo, como em nenhum momento incluímos um cão neste índice, teremos aqui uma exceção nos informando de que “o valor de retorno na posição é nulo, ou a posição do vetor é válida, mas está vazia”. O texto original da exceção é:

Exception in thread "main" java.lang.NullPointerException: Cannot invoke "Vetor.Cao.printCao()" because the return value of "Vetor.Dono.getPet(int)" is null

Agora, e se em vez das alterações acima alterássemos a linha 46 de forma que o valor do índice que pipoca deve ter no vetor pets seja 20? No caso, a linha deverá ficar assim:

```
linha 59 maria.addPet(20, pipoca);
```

Aqui, como o índice 20 está fora dos limites do vetor pets. Afinal, este vetor possui somente os índices 0, 1 e 2 (totalizando assim 3 objetos de Cao). Portanto, teremos uma outra exceção do tipo “índice fora dos limites do vetor”:

*Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
Index 20 out of bounds for length 3*

Listas

Vimos brevemente alguns exemplos nas semanas anteriores utilizando **ArrayList**. Agora, qual é a diferença entre **int[]** e **ArrayList<Integer>**?

Vetores como o **int[]** são um arrays de tamanho fixo, o que significa que, uma vez definido, você não pode alterar seu tamanho. Já **ArrayList** é uma classe da coleção Java que pode armazenar elementos dinamicamente e permite adicionar ou remover elementos após a sua criação, o que o torna mais flexível em comparação com o array tradicional. Para usar o **ArrayList** sempre precisaremos adicionar a linha “import

java.util.ArrayList;” ao início dos arquivos que o usem. O motivo disso está no fato de que o **ArrayList** faz parte de uma biblioteca específica do Java (`java.util`) que possui a sua implementação.

Atenção: em Java, as listas não gostam de tipos primitivos. Diferente dos vetores (`int[]`), o `ArrayList` só consegue guardar objetos. Isso significa que você não pode escrever `ArrayList<int>` ou `ArrayList<double>`. Para resolver isso, o Java possui classes especiais chamadas wrappers (envelopes), que transformam primitivos em objetos: `int` vira `Integer`; `double` vira `Double`; e `boolean` vira `Boolean`. Logo, em vez de `ArrayList<int>` usaríamos `ArrayList<Integer>`, por exemplo.

A forma de acesso dos `ArrayLists` é muito parecida com o dos vetores. Veja:



Fonte: Autores (2020).

O que acha de implementarmos um exemplo?



EXEMPLO

Exemplo de aplicação 5: crie uma classe em Java chamada **ListaCores**. Ela precisa de um `ArrayList` de strings. Inclua

quatro cores nesta lista. Depois, crie operações para ler, editar e remover elementos desta lista.

</>

```
1 import java.util.ArrayList;
2
3 public class ListaCores {
4     public static void main(String[] args) {
5         int i;
6         // Declara a instancia o ArrayList cores
7         ArrayList<String> cores = new ArrayList<String>();
8         cores.add("Azul");    // Inclui elemento no ArrayList
9         cores.add("Verde");   // Inclui elemento no ArrayList
10        cores.add("Vermelho"); // Inclui elemento no ArrayList
11        cores.add("Amarelo");  // Inclui elemento no ArrayList
12
13        // Loop para percorrer a lista, elemento por elemento
14        for (i = 0; i < cores.size(); i++) // imprime cada elemento
15            System.out.println((i + 1) + "º " + cores.get(i));
16
17        // ALTERA elemento da lista:
18        cores.set(1, "Pink"); // altera elemento na posição 1 para "Pink"
19    }
```

O resultado deste código será:

```
1º) Azul
2º) Verde
3º) Vermelho
```

```
4º) Amarelo
----
1º) Azul
2º) Pink
3º) Vermelho
4º) Amarelo
----
1º) Azul
2º) Pink
3º) Vermelho
----
Tamanho da lista = 0

Process finished with exit code 0
```

Agora, o que aquele código faz? Vamos entender juntos:

1. **ArrayList cores = new ArrayList();** Este é o momento em que um ArrayList chamado **cores** é criado. Este ArrayList é projetado para conter objetos do tipo string.
2. Em seguida, quatro cores são adicionadas à lista usando o método **.add()**. Portanto, a lista **cores** agora contém "azul", "verde", "vermelho" e "amarelo", nesta ordem.
3. O primeiro loop **for** é usado para imprimir cada cor da lista. O método **.size()** é usado para determinar o número de elementos no ArrayList, e o método **.get(i)** é usado para acessar o elemento na posição i.
4. O método **.set(1, "Pink");** é usado para alterar o elemento na posição 1 (lembrando que a contagem começa em 0, então este é o segundo elemento) de "Verde" para "Pink".
5. O segundo loop **for** (conhecido como loop "for-each") é usado para imprimir novamente cada cor da lista, mostrando que "Verde" foi substituído por "Pink".
6. O método **.remove(3);** é usado para remover o elemento na posição 3 (quarto elemento, neste caso, "amarelo") da lista.
7. O terceiro loop **for** imprime a lista novamente, mostrando que "Amarelo" foi removido.
8. Finalmente, o método **.clear();** é usado para remover todos os elementos da lista, e o tamanho da lista é impresso para confirmar que está vazia (tamanho 0).

Você deve ter percebido elementos como **.clear()**, **.remove()**, **.add()** e afins, não é? Eles são métodos da classe `ArrayList`, muito úteis e que tornam esse recurso poderoso, provendo muita flexibilidade para a manipulação de uma **coleção de objetos**. Eles lembram muito métodos de manipulação de listas do Python, como o **.append()**, por exemplo.

1. <code>add(Object obj)</code>	<code>// insere um objeto no fim da lista</code>
2. <code>add(int index, Object obj)</code>	<code>// insere um objeto na posição especificada</code>
3. <code>remove(Object obj)</code>	<code>// remove da lista o objeto especificado</code>
4. <code>remove(int index)</code>	<code>// remove da lista o objeto na posição especificada</code>
5. <code>set(int index, Object obj)</code>	<code>// atualiza o objeto na posição especificada</code>
6. <code>int indexOf(Object obj)</code>	<code>// retorna a posição do objeto especificado</code>
7. <code>Object get(int index)</code>	<code>// retorna o objeto na posição especificada</code>
8. <code>int size()</code>	<code>// retorna o tamanho da lista</code>
9. <code>boolean contains(Object obj)</code>	<code>// verifica se o objeto passado está na lista</code>
10. <code>clear()</code>	<code>// remove todos os objetos da lista</code>

Referência para consulta: https://www.w3schools.com/java/java_arraylist.asp.

| Utilização de objeto e referência de objeto

Agora, vamos entender algumas formas diferentes de usarmos objetos. Relembrando: um **objeto é uma instância de uma classe** e uma **classe pode ter vários objetos**, ou várias **instâncias**. Quando precisamos acessar esses objetos, utilizamos sua referência, que é o nome da variável do objeto que instanciamos, nos nossos códigos.

Para **instanciar uma classe**, ou **criar um objeto**, em Java, usamos a palavra-chave `new`, como demonstrado a seguir, em que o objeto da classe dono tem o **nome de variável**, ou **referência**, igual a maria:

```
Dono maria = new Dono("Maria"); // Maria é referência à instância de Dono
```



IMPORTANTE

Em Java, um **objeto existe** apenas quando é **instanciado** com a palavra-chave `new`.

Podemos e devemos utilizar os objetos e suas referências de diferentes formas. Veremos, a partir de agora, alguns exemplos de como poderemos fazer isto.

Objeto utilizado como atributo

Espero que você não tenha fechado aquele código das classes cao, dono e cauda. Abra novamente a classe cao. Na **linha 2**, temos o atributo `meuDono`, do tipo classe dono. Da mesma forma, na **linha 7**, temos o atributo `minhaCauda`, do tipo classe cauda.

O nome dos atributos `meuDono` e `minhaCauda` também são referências a esses objetos. Isso significa que um objeto da classe Cao faz referência a objetos das classes dono e cauda:

</>

```
1 public class Cao {  
2     private Dono meuDono;    // atributo = objeto da classe Dono  
...  
7     private Cauda minhaCauda; // atributo = objeto da classe Cauda  
...  
...
```

**DICA**

Quando dizemos que “um objeto da classe cao faz referência a objetos das classes dono e cauda” queremos dizer que um objeto da classe cão “sabe” quem é seu dono e “sabe” que tem uma cauda, e pode interagir com ambos de maneira programada.

Objeto utilizado como variável local

Para que um objeto de cao fique completo, precisamos instanciar dono e cauda. Isso é feito na classe dono, nas **linhas 27 a 31**, da classe dono conforme havíamos implementado no exercício anterior:

</>

```
26     public static void main(String[] args) {  
27         Dono maria = new Dono("Maria");  
28         Cao pipoca = new Cao("Pipoca", "Beagle", "Fêmea", 3,  
29             "Enrolada", "Pêlo curtinho");  
30         maria.setPet(pipoca); // associa Maria com Pipoca  
31         pipoca.setMeuDono(maria); // associa Pipoca com Maria  
...  
38     }
```

Observe que as variáveis `maria` e `pipoca`, que são **referências a objetos** das classes `Dono` e `Cao` respectivamente, também são **variáveis locais** pois existem apenas dentro do método `main`: ou seja, elas somente são válidas entre as **linhas 27 a 38** na classe `Dono`.



DICA

Variáveis locais apenas são **acessíveis (ou visíveis)** dentro do escopo em que foram criados, como dentro dos limitadores “{” e “}” de um método.

Objeto utilizado como parâmetro (argumento)

Novamente na classe `Dono`, vemos o método `Dono.setPet` que espera receber um objeto da classe `Cao` como **argumento ou parâmetro**. Mais especificamente, nas linhas 9 e 10.

</>

```
9 | public void setPet(Cao pet) { // argumento = pet
10 |     this.pet = pet;    // atributo recebe o argumento
11 | }
```

Da mesma forma, na classe cao o método `Cao.setMeuDono` espera receber um objeto da classe dono como **argumento ou parâmetro**:

</>

```
17 | public void setMeuDono(Dono meuDono) { // argumento = meuDono
18 |     this.meuDono = meuDono;    // atributo recebe o argumento
20 | }
```

Esses métodos `Dono.setPet` e `Cao.setMeuDono` apenas podem ser invocados se existirem os objetos que serão usados como parâmetros. Veja as suas **invocações** no exemplo da classe dono nas **linhas 30 e 31**. Aqui, os objetos de classes dono e cao são usados como **parâmetros** (ou **argumentos**) de método:

</>

```
26 public static void main(String[] args) {
27     Dono maria = new Dono("Maria");
28     Cao pipoca = new Cao("Pipoca", "Beagle", "Fêmea", 3,
29                          "Enrolada", "Pêlo curtinho");
30     maria.setPet(pipoca);    // associa Maria com Pipoca
31     pipoca.setMeuDono(maria); // associa Pipoca com Maria
...
38 }
```

Primeiramente, instanciamos maria e pipoca (linhas 27 e 28). Somente depois disso, poderemos utilizá-los como parâmetros nas linhas 30 e 31.

Objeto utilizado como retorno de método

Também, podemos usar os objetos como retorno de função. Vejamos um exemplo na classe cao: o método `Cao.getNomeCao` **retorna** o **valor do atributo** `nomeCao`, que é do tipo `string`. Em Java, `strings` são **classes**. Logo, `nomeCao` faz referência a um objeto da classe `string`.

</>

```
22 | public String getNomeCao() {  
23 |     return nomeCao;  
24 | }
```

Relembrando: Um método tem a sua **assinatura** definida como (mais detalhes na Unidade 2).

```
tipo nomeMetodo (tipo parametro1, tipo parametro2, ..., tipo parametroN) {  
    // corpo do método  
}
```

Antes do nome do método temos o tipo de dado que o método retorna. Neste exemplo, o tipo de dado é uma **instância** (objeto) da classe `string`. Logo, o método tem que ter obrigatoriamente a palavra-chave `return`, seguido do valor que será retornado pela invocação (chamada do método).

Exercícios de fixação

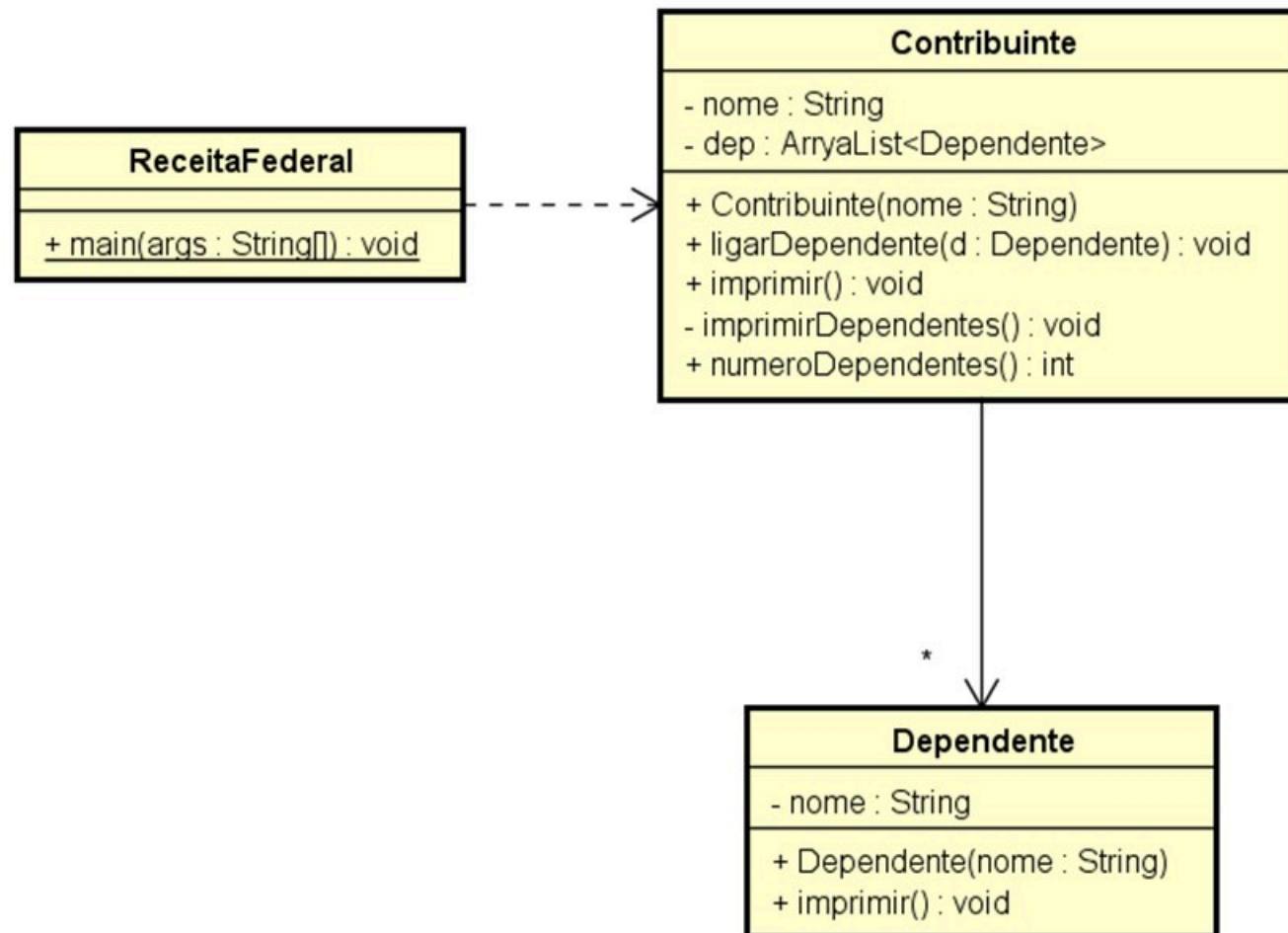


EXERCÍCIO

1. Crie uma classe **carro** com atributos para "marca", "modelo", "ano" e "placa". Em seguida, crie uma classe **MotoristaUber** que possui um objeto **carro** como um de seus atributos, além de atributos para "nome" e

- "idMotorista". Crie métodos para definir e obter esses atributos.
2. Crie uma classe **post** com atributos para "texto", "quantidadeDeCurtidas" e "quantidadeDeCompartilhamentos". Em seguida, crie uma classe **usuario** que contém um ArrayList de **post** para representar o feed de notícias do usuário.
 3. Crie uma classe **produto** com atributos para "nome", "preco", "quantidadeEmEstoque". Em seguida, crie uma classe **CarrinhoDeCompras** que contém um ArrayList de **produto**. Adicione métodos à classe **CarrinhoDeCompras** para adicionar produtos ao carrinho, remover produtos e calcular o total do carrinho.
 4. Crie uma classe **tweet** com atributos para "texto", "usuario" e "retweets". Em seguida, crie uma classe **usuario** que contém um ArrayList de **tweet** para representar os tweets de um usuário. Crie métodos para postar um novo tweet e para retweetar um tweet existente.
 5. Crie uma classe **local** com atributos para "nome", "latitude" e "longitude". Em seguida, crie uma classe **rota** que contém um ArrayList de **local** para representar uma rota de um ponto de origem a um destino. Adicione métodos à classe **rota** para adicionar um novo local à rota e para calcular a distância total da rota.
 6. **Desafio:** transforme o diagrama UML a seguir em um algoritmo em Java.

#ParaTodosVerem 



Fonte: Autores (2020).

</>

```
1  class Carro {
2      private String marca;
3      private String modelo;
4      private int ano;
5      private String placa;
6
7      public Carro(String marca, String modelo, int ano, String placa) {
8          this.marca = marca;
9          this.modelo = modelo;
10         this.ano = ano;
11         this.placa = placa;
12     }
13
14     // Getters
15     public String getMarca() {
16         return this.marca;
17     }
18
19     public String getModelo() {
```

Exercício 2

</>

```
1 import java.util.ArrayList;
2
3 class Post {
4     private String texto;
5     private int quantidadeDeCurtidas;
6     private int quantidadeDeCompartilhamentos;
7
8     public Post(String texto) {
9         this.texto = texto;
10        this.quantidadeDeCurtidas = 0;
11        this.quantidadeDeCompartilhamentos = 0;
12    }
13
14    public void curtir() {
15        this.quantidadeDeCurtidas++;
16    }
17
18    public void compartilhar() {
19        this.quantidadeDeCompartilhamentos++;
20    }
21 }
```

Exercício 3

</>

```
1 import java.util.ArrayList;
2
3 class Produto {
4     private String nome;
5     private double preco;
6     private int quantidadeEmEstoque;
7
8     public Produto(String nome, double preco, int quantidadeEmEstoque) {
9         this.nome = nome;
10        this.preco = preco;
11        this.quantidadeEmEstoque = quantidadeEmEstoque;
12    }
13
14    public String getNome() {
15        return nome;
16    }
17
18    public double getPreco() {
19        return preco;
```

Exercício 4

</>

```
1 import java.util.ArrayList;
2
3 class Tweet {
4     private String texto;
5     private String usuario;
6     private int retweets;
7
8     public Tweet(String texto, String usuario) {
9         this.texto = texto;
10        this.usuario = usuario;
11        this.retweets = 0;
12    }
13
14    public String getTexto() {
15        return texto;
16    }
17
18    public String getUsuario() {
19        return usuario;
```

Exercício 5

</>

```
1 import java.util.ArrayList;
2
3 class Local {
4     private String nome;
5     private double latitude;
6     private double longitude;
7
8     public Local(String nome, double latitude, double longitude) {
9         this.nome = nome;
10        this.latitude = latitude;
11        this.longitude = longitude;
12    }
13
14    public String getNome() {
15        return nome;
16    }
17
18    public double getLatitude() {
19        return latitude;
```

Exercício 6

Arquivo **Dependente.java**

</>

```
1 public class Dependente {  
2     private String nome;  
3  
4     public Dependente(String nome) {  
5         this.nome = nome;  
6     }  
7     public void imprimir() {  
8         System.out.println ("Dependente: " + nome);  
9     }  
10 }
```

Arquivo Contribuinte.java

</>

```
1 import java.util.ArrayList;
2
3 public class Contribuinte {
4     private String nome;
5     private ArrayList <Dependente> dep;
6     public Contribuinte (String nome) {
7         this.nome = nome;
8         dep = new ArrayList <Dependente>();
9     }
10    public void ligarDependente (Dependente d) {
11        dep.add (d);
12    }
13    public void imprimir () {
14        System.out.println("Contribuinte: " + this.nome);
15        imprimirDependentes();
16    }
17    private void imprimirDependentes () {
18        for (Dependente d : dep) {
19            d.imprimir();
```

Arquivo **ReceitaFederal.java**

</>


```
1 public class ReceitaFederal {
2
3     public static void main(String[] args) {
4         Contribuinte julia = new Contribuinte ( "Julia");
5         Dependente jorge = new Dependente ( "Jorge");
6         Dependente sandra = new Dependente ( "Sandra");
7
8         julia.ligarDependente(jorge);
9         julia.ligarDependente(sandra);
10        julia.imprimir();
11        System.out.println("Numero de dependentes: " +
12                           julia.numeroDependentes ( ) + "\n");
13
14        Contribuinte leonardo = new Contribuinte ("Leonardo");
15        Dependente marta = new Dependente ("Marta");
16        Dependente diego = new Dependente ("Diego");
17        Dependente claudia = new Dependente ("Claudia");
18        leonardo.ligarDependente(marta);
19        leonardo.ligarDependente(diego);
```

| Referências Bibliográficas

GODOY, V. **Programação orientada a objetos I**. Curitiba: IESDE, 2019.

HORSTMANN, C. S.; CORNELL, G. **Core Java – volume I**. 8. ed. São Paulo: Pearson, 2010.

